

Tower Induction and Interaction Trees: Experience Report

Anonymous Author(s)

Abstract

Parameterized coinduction, or *paco*, improves on *Rocq*'s awkward *cofix* tactic for low-level coinductive reasoning. But *paco* still relies on many interlocking definitions and a wide array of library tactics that can make proofs complex. The *rocq-coinduction* library, which mechanizes the tower induction principle of Schäfer and Smolka, promises shorter and more uniform proofs: tower induction is the single proof method for both coinduction and soundness for up-to techniques, and it requires fewer tactics than *paco* to employ.

We assessed that promise at scale by porting the coinductive-proof infrastructure of Interaction Trees (ITrees), a large coinduction-heavy library, from *paco* to *rocq-coinduction*. We found a mixed result: proof text decreased by 12%, but tactic text nearly tripled. *rocq-coinduction* required significant client-side plumbing around its exposed tactics and contained several usability bugs that required verbose workarounds.

Development also stalled because *rocq-coinduction*'s tactics were written in OCaml over complex *Rocq* infrastructure, which made them difficult to debug.

Observing that *rocq-coinduction* was theoretically rich but awkward in implementation, we rewrote its tactic layer from scratch. The new implementation is *Rocq*-native, replacing the OCaml plugin with Gallina and Ltac, and is 65% smaller than the code it supersedes while exporting more expressive tactics and removing the limitations above. Rerunning the ITrees port against the reimplemented library keeps the proof reduction and takes tactic text from 178% above the *paco* baseline to 44.5% below it. These results suggest that the re-engineered tower-based coinduction library is not only a theoretical simplification but a practical improvement over *paco* for large coinductive developments.